

POORNIMA COLLEGE OF ENGINEERING, JAIPUR

Department of Information Technology

Analysis of Algorithms (AoA) Lab

Course Code: 245CY4-21

Practical Record — Experiments 1 to 6

Name: Shourya Sharma

Roll No.: PCEA24IT051

S.No.	Experiment
1	Quick Sort with execution time
2	Merge Sort with execution time
3(a)	Topological ordering of a DAG
3(b)	Transitive closure using Warshall's Algorithm
4	DFS and BFS traversal
5	Dijkstra's shortest path algorithm
6	Minimum spanning tree using Prim's algorithm

Experiment 1

Aim

Sort a given set of elements using the Quicksort method and determine the time required to sort the elements. Repeat the experiment for different values of n . The elements can be read from a file or generated using the random number generator.

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C
- **Header files:** `stdio.h`, `stdlib.h` (for `rand()`), `time.h` (for `clock()`)

Theory

Quick Sort is a sorting algorithm based on the **Divide and Conquer** technique. It was developed by Tony Hoare.

It works in three steps:

1. **Divide:** choose an element as the **pivot** (here, the last element) and **partition** the array so that all elements smaller than or equal to the pivot come before it and all larger elements come after it. The pivot is now in its final sorted position.
2. **Conquer:** recursively apply Quick Sort to the left and right sub-arrays.
3. **Combine:** no work is needed, because the array is sorted in place.

The partition method used here is the **Lomuto partition scheme**.

Measuring time: the `clock()` function from `time.h` returns processor time. The time taken is $(\text{end} - \text{start}) / \text{CLOCKS_PER_SEC}$ seconds. `rand() % 1000` generates random numbers between 0 and 999, and `srand(time(NULL))` gives a different sequence on each run.

Complexity:

Case	Time Complexity	When it occurs
Best Case	$O(n \log n)$	Pivot divides the array into two equal halves
Average Case	$O(n \log n)$	Random input
Worst Case	$O(n^2)$	Array already sorted or reverse sorted

Space complexity: $O(\log n)$ for the recursion stack in the average case. Quick Sort is **in-place** but **not stable**.

Algorithm

QUICKSORT(arr, low, high):

1. If $\text{low} < \text{high}$:
 - `p = PARTITION(arr, low, high)`
 - `QUICKSORT(arr, low, p - 1)`
 - `QUICKSORT(arr, p + 1, high)`

PARTITION(arr, low, high):

1. `pivot = arr[high]`, `i = low - 1`
2. For `j = low` to `high - 1`: if `arr[j] ≤ pivot`, increment `i` and swap `arr[i]` with `arr[j]`.

3. Swap `arr[i + 1]` with `arr[high]`.
4. Return `i + 1`.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Function to swap two elements
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] <= pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Quick Sort function
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
```

```

    srand(time(NULL));

    // Generate random numbers
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 1000;

    printf("\nArray before sorting:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\n\nArray after sorting:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n\nExecution Time = %lf seconds\n", time_taken);
    return 0;
}

```

Output

```

Enter number of elements: 10

Array before sorting:
65 12 98 45 21 76 34 11 89 55

Array after sorting:
11 12 21 34 45 55 65 76 89 98

Execution Time = 0.000012 seconds

```

Time for different values of n (to plot the graph):

Number of Elements (n)	Time (seconds)
1000	
5000	
10000	
20000	
50000	

Plot **n on the X-axis** and **time on the Y-axis**. The curve rises slightly faster than a straight line, which shows the $n \log n$ growth.

Result

Quick Sort was successfully implemented and executed. The random elements were sorted in ascending order, and the execution time was measured for different values of n. The time increases in proportion to $n \log n$, which confirms the average-case complexity of Quick Sort.

Experiment 2

Aim

To implement Merge Sort and determine the execution time for sorting the given elements.

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C
- **Header files:** stdio.h, stdlib.h, time.h

Theory

Merge Sort is a sorting algorithm based on the **Divide and Conquer** technique. It was invented by John von Neumann.

It works in three steps:

1. **Divide:** split the array into two halves at the middle index $m = (l + r) / 2$.
2. **Conquer:** recursively sort both halves until each sub-array has one element. A single element is already sorted.
3. **Combine:** merge the two sorted halves into one sorted array by repeatedly comparing the smallest remaining elements of each half.

The main work is done in the **merge** step, which uses two temporary arrays L[] and R[].

Recurrence relation: $T(n) = 2T(n/2) + O(n)$, which solves to **$O(n \log n)$** .

Complexity:

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Space complexity: $O(n)$ for the temporary arrays. Merge Sort is **stable** but **not in-place**.

Merge Sort vs Quick Sort: Merge Sort always takes $O(n \log n)$ time even in the worst case, but it needs extra memory. Quick Sort sorts in place but can degrade to $O(n^2)$.

Algorithm

MERGESORT(arr, l, r):

1. If $l < r$:
 - $m = (l + r) / 2$
 - MERGESORT(arr, l, m)
 - MERGESORT(arr, m + 1, r)
 - MERGE(arr, l, m, r)

MERGE(arr, l, m, r):

1. Copy arr[l..m] into L[] and arr[m+1..r] into R[].
2. Compare L[i] and R[j]. Copy the smaller one into arr[k] and advance that index.
3. Copy any remaining elements of L[] and R[] into arr.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```

int arr[n];
printf("Enter elements:\n");
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

clock_t start, end;
start = clock();
mergeSort(arr, 0, n - 1);
end = clock();

printf("\nSorted Array:\n");
for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

double time = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nExecution Time = %lf seconds\n", time);

return 0;
}

```

Output

```

Enter number of elements: 8
Enter elements:
45 12 78 23 56 10 90 34

Sorted Array:
10 12 23 34 45 56 78 90
Execution Time = 0.000012 seconds

```

Result

Merge Sort was successfully implemented and executed. The given elements were sorted in ascending order, and the execution time was measured using the clock() function. Merge Sort gives $O(n \log n)$ time in all cases.

Experiment 3(a)

Aim

To obtain the topological ordering of vertices in a Directed Acyclic Graph (DAG).

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C

Theory

A **Directed Acyclic Graph (DAG)** is a directed graph that contains **no cycles**.

Topological sorting of a DAG is a linear ordering of its vertices such that for every directed edge $u \rightarrow v$, vertex u comes **before** vertex v in the ordering. Topological ordering is possible **only for DAGs**. If the graph has a cycle, no valid ordering exists. A DAG can have more than one valid topological order.

In-degree of a vertex is the number of edges coming into it.

This program uses the **source removal method (Kahn's algorithm idea)**:

1. Compute the in-degree of every vertex.
2. Repeatedly pick a vertex with in-degree 0 (a **source**), print it, and remove it by reducing the in-degree of all its neighbours by 1.
3. Mark the removed vertex with in-degree -1 so it is not picked again.
4. If at some point no vertex has in-degree 0 but vertices remain, the graph contains a **cycle**.

Applications: task scheduling, course prerequisite ordering, compilation order of files, and resolving dependencies in build systems.

Time complexity: $O(V^2)$ with an adjacency matrix, or $O(V + E)$ with an adjacency list.

Space complexity: $O(V)$ for the in-degree array.

Algorithm

1. Read n and the adjacency matrix. For every edge $i \rightarrow j$, increment $\text{indegree}[j]$.
2. Set $\text{count} = 0$.
3. While $\text{count} < n$:
 - Set $\text{found} = 0$.
 - For each vertex i with $\text{indegree}[i] = 0$: print i , set $\text{indegree}[i] = -1$, decrement $\text{indegree}[j]$ for every edge $i \rightarrow j$, set $\text{found} = 1$, and increment count .
 - If $\text{found} = 0$, print "Cycle detected" and stop.
4. Stop.

Code

```

#include <stdio.h>

int graph[10][10], indegree[10], n;

void topologicalSort()
{
    int count = 0;

    while (count < n)
    {
        int found = 0;
        for (int i = 0; i < n; i++)
        {
            if (indegree[i] == 0)
            {
                printf("%d ", i);
                indegree[i] = -1;

                for (int j = 0; j < n; j++)
                    if (graph[i][j])
                        indegree[j]--;

                found = 1;
                count++;
            }
        }
        if (!found)
        {
            printf("\nCycle detected.");
            return;
        }
    }
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        indegree[i] = 0;
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
            if (graph[i][j])
                indegree[j]++;
        }
    }

    printf("Topological Order:\n");
    topologicalSort();
    return 0;
}

```

Output

```
Enter number of vertices: 6
Enter adjacency matrix:
0 1 1 0 0 0
0 0 0 1 1 0
0 0 0 1 0 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological Order:
0 1 2 3 4 5
```

Result

The topological ordering of the given DAG was successfully obtained using the source removal method. The order 0 1 2 3 4 5 satisfies the condition that for every edge $u \rightarrow v$, u appears before v .

Experiment 3(b)

Aim

To compute the transitive closure of a directed graph using Warshall's Algorithm.

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C

Theory

The **transitive closure** of a directed graph with n vertices is an $n \times n$ boolean matrix T , where $T[i][j] = 1$ if there is a path (direct or indirect) from vertex i to vertex j , and 0 otherwise. It shows which vertices are **reachable** from which.

Warshall's Algorithm computes the transitive closure using **Dynamic Programming**. It builds a sequence of matrices $R^{(0)}, R^{(1)}, \dots, R^{(n)}$, where $R^{(0)}$ is the adjacency matrix and $R^{(k)}$ allows paths that use only vertices 0 to k as intermediate vertices.

The key formula is:

$$R^{(k)}[i][j] = R^{(k-1)}[i][j] \text{ OR } (R^{(k-1)}[i][k] \text{ AND } R^{(k-1)}[k][j])$$

This means there is a path from i to j if one already existed, or if there is a path from i to k and from k to j .

Applications: reachability queries in networks, database query processing, and dependency analysis in compilers.

Time complexity: $O(n^3)$, because of three nested loops.

Space complexity: $O(n^2)$ for the matrix. The update is done in place in the same matrix.

Algorithm

1. Read n and the adjacency matrix a .
2. For $k = 0$ to $n - 1$, for $i = 0$ to $n - 1$, for $j = 0$ to $n - 1$:
 $a[i][j] = a[i][j] \text{ OR } (a[i][k] \text{ AND } a[k][j])$
3. Print matrix a as the transitive closure.
4. Stop.

Code

```

#include <stdio.h>

int main()
{
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int a[n][n];
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = a[i][j] || (a[i][k] && a[k][j]);

    printf("\nTransitive Closure Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
    return 0;
}

```

Output

```

Enter number of vertices: 4
Enter adjacency matrix:
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0

Transitive Closure Matrix:
0 1 1 1
0 0 1 1
0 0 0 1
0 0 0 0

```

Result

Warshall's algorithm was successfully implemented and executed. The transitive closure matrix shows that vertex 0 can reach 1, 2, and 3, vertex 1 can reach 2 and 3, and vertex 2 can reach 3, which matches the path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ in the given graph.

Experiment 4

Aim

Write a program to demonstrate Depth First Search (DFS) and Breadth First Search (BFS) on a graph.

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C

Theory

A **graph** $G = (V, E)$ is a set of vertices V connected by edges E . It can be stored as an **adjacency matrix**, where $\text{graph}[i][j] = 1$ if there is an edge between vertex i and vertex j , and 0 otherwise. Graph traversal means visiting every vertex of the graph in a systematic way.

Breadth First Search (BFS) starts from a source vertex and visits all of its neighbours first, then the neighbours of those neighbours, level by level. It uses a **queue** (FIFO). BFS is used for shortest paths in unweighted graphs, level-order traversal, and finding connected components.

Depth First Search (DFS) starts from a source vertex and goes as deep as possible along one branch before backtracking. It uses a **stack**, either explicitly or through recursion. DFS is used for checking connectivity, cycle detection, topological sorting, and path finding.

Connectivity check: if DFS from vertex 0 visits every vertex, the graph is connected. If any vertex is left unvisited, the graph is not connected.

Time complexity: $O(V^2)$ for both BFS and DFS with an adjacency matrix. With an adjacency list it is $O(V + E)$.

Space complexity: $O(V)$ for the visited array and the queue or recursion stack.

Algorithm

BFS(start):

1. Mark all vertices as unvisited.
2. Insert start into the queue and mark it visited.
3. While the queue is not empty:
 - Remove the front vertex current and print it.
 - For every unvisited neighbour i of current, mark it visited and insert it into the queue.
4. Stop.

DFS(vertex):

1. Mark vertex as visited.
2. For every unvisited neighbour i of vertex, call DFS(i).
3. Stop.

Code

```

#include <stdio.h>
#define MAX 20

int graph[MAX][MAX];
int visited[MAX];
int n;

// BFS
void BFS(int start) {
    int queue[MAX];
    int front = 0, rear = 0;
    int i, current;

    for (i = 0; i < n; i++)
        visited[i] = 0;

    queue[rear++] = start;
    visited[start] = 1;

    printf("Nodes reachable from node %d using BFS: ", start);
    while (front < rear) {
        current = queue[front++];
        printf("%d ", current);
        for (i = 0; i < n; i++) {
            if (graph[current][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("\n");
}

// DFS
void DFS(int vertex) {
    int i;
    visited[vertex] = 1;
    for (i = 0; i < n; i++) {
        if (graph[vertex][i] == 1 && visited[i] == 0) {
            DFS(i);
        }
    }
}

int main() {
    int i, j, start;
    int connected = 1;

```

```

printf("Enter number of vertices: ");
scanf("%d", &n);

printf("Enter adjacency matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        scanf("%d", &graph[i][j]);

printf("\nEnter starting node for BFS (0 to %d): ", n - 1);
scanf("%d", &start);
BFS(start);

for (i = 0; i < n; i++)
    visited[i] = 0;

DFS(0);
for (i = 0; i < n; i++) {
    if (visited[i] == 0) {
        connected = 0;
        break;
    }
}

if (connected)
    printf("The graph is connected\n");
else
    printf("The graph is not connected\n");

return 0;
}

```

Output

```

Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0

Enter starting node for BFS (0 to 4): 0
Nodes reachable from node 0 using BFS: 0 1 2 3 4
The graph is connected

```

Result

The program for BFS and DFS traversal was successfully implemented and executed. BFS printed all nodes reachable from the source node, and DFS was used to verify that the given graph is connected.

Experiment 5

Aim

Write a program to demonstrate Dijkstra's algorithm for finding the shortest path from a single source to all other vertices.

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C

Theory

Dijkstra's algorithm solves the **single-source shortest path** problem for a weighted graph with **non-negative edge weights**. It finds the minimum distance from a source vertex to every other vertex.

It is a **greedy algorithm**: at every step it picks the unvisited vertex with the smallest known distance, marks it as finalised, and then updates the distances of its neighbours. This update step is called **relaxation**:

if $d[u] + \text{cost}(u, v) < d[v]$, then $d[v] = d[u] + \text{cost}(u, v)$

A large value (INF = 999) represents "no direct edge" between two vertices.

Limitation: Dijkstra's algorithm does not work correctly with negative edge weights. Bellman-Ford is used in that case.

Applications: GPS and map navigation, network routing protocols (OSPF), and flight or transport route planning.

Time complexity: $O(V^2)$ with an adjacency matrix, or $O((V + E) \log V)$ with a min-heap.

Space complexity: $O(V)$ for the distance and visited arrays.

Algorithm

1. Read the number of vertices n , the cost matrix a , and the source s .
2. Set $d[i] = a[s][i]$ for all i . Set $d[s] = 0$ and mark s as visited.
3. Repeat $n - 1$ times:
 - Find the unvisited vertex u with minimum $d[u]$.
 - Mark u as visited.
 - For every unvisited vertex i , if $d[u] + a[u][i] < d[i]$, set $d[i] = d[u] + a[u][i]$.
4. Print $d[i]$ for all vertices.
5. Stop.

Code

```

#include <stdio.h>
#define INF 999

int main() {
    int n, a[10][10], d[10], v[10] = {0};
    int i, j, k, min, u, s;

    printf("Enter vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    printf("Enter source: ");
    scanf("%d", &s);

    for (i = 0; i < n; i++)
        d[i] = a[s][i];
    d[s] = 0;
    v[s] = 1;

    for (k = 1; k < n; k++) {
        min = INF;
        for (i = 0; i < n; i++)
            if (!v[i] && d[i] < min) {
                min = d[i];
                u = i;
            }
        v[u] = 1;
        for (i = 0; i < n; i++)
            if (!v[i] && d[u] + a[u][i] < d[i])
                d[i] = d[u] + a[u][i];
    }

    printf("\nShortest paths:\n");
    for (i = 0; i < n; i++)
        printf("%d -> %d = %d\n", s, i, d[i]);

    return 0;
}

```

Output

```

Enter vertices: 4
Enter adjacency matrix:
0 4 2 999
4 0 999 1
2 999 0 5
999 1 5 0
Enter source: 0

Shortest paths:
0 -> 0 = 0
0 -> 1 = 4
0 -> 2 = 2
0 -> 3 = 5

```

Result

Dijkstra's algorithm was successfully implemented and executed. The shortest distances from source vertex 0 to all other vertices were computed correctly. For example, $0 \rightarrow 3 = 5$

through the path $0 \rightarrow 1 \rightarrow 3$.

Experiment 6

Aim

Find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

Requirements

- **Hardware:** PC with minimum 4 GB RAM
- **Software:** Windows/Linux OS, C compiler (GCC / Turbo C / Code::Blocks)
- **Language:** C

Theory

A **spanning tree** of a connected undirected graph is a subgraph that includes **all V vertices** connected with exactly **V – 1 edges** and contains **no cycle**. A **Minimum Spanning Tree (MST)** is the spanning tree whose total edge weight is the smallest possible.

Prim's algorithm is a **greedy algorithm** that builds the MST one vertex at a time. It starts from any vertex (here vertex 0). At each step it picks the **minimum-weight edge** that connects a vertex already in the tree to a vertex not yet in the tree, and adds that edge and vertex to the tree. This repeats until all vertices are included.

Difference from Kruskal's algorithm: Prim's grows a single tree from one vertex. Kruskal's sorts all edges and adds the smallest edges that do not form a cycle.

Applications: designing low-cost networks for cables, roads, pipelines, and electrical wiring, and cluster analysis.

Time complexity: $O(V^3)$ for this simple matrix implementation, $O(V^2)$ with the key-array method, or $O(E \log V)$ with a min-heap.

Space complexity: $O(V)$ for the visited array.

Algorithm

1. Read the number of vertices n and the cost matrix a .
2. Mark vertex 0 as visited. Set cost = 0.
3. Repeat $n - 1$ times:
 - Set min = INF.
 - For every visited vertex i and every unvisited vertex j , if $a[i][j] < \text{min}$, set $\text{min} = a[i][j]$, $x = i$, $y = j$.
 - Print edge (x, y) with weight min.
 - Add min to cost and mark y as visited.
4. Print the total minimum cost.
5. Stop.

Code

```

#include <stdio.h>
#define INF 999

int main() {
    int n, a[10][10], v[10] = {0};
    int i, j, k, min, x, y, cost = 0;

    printf("Enter vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    v[0] = 1;
    printf("\nEdges in MST:\n");

    for (k = 1; k < n; k++) {
        min = INF;
        for (i = 0; i < n; i++)
            if (v[i])
                for (j = 0; j < n; j++)
                    if (!v[j] && a[i][j] < min) {
                        min = a[i][j];
                        x = i;
                        y = j;
                    }
        printf("%d - %d = %d\n", x, y, min);
        cost += min;
        v[y] = 1;
    }

    printf("Minimum cost = %d\n", cost);
    return 0;
}

```

Output

```

Enter vertices: 4
Enter adjacency matrix:
0 2 3 999
2 0 1 4
3 1 0 5
999 4 5 0

Edges in MST:
0 - 1 = 2
1 - 2 = 1
1 - 3 = 4
Minimum cost = 7

```

Result

Prim's algorithm was successfully implemented and executed. The minimum spanning tree of the given graph was found with edges (0-1), (1-2), and (1-3), and a total minimum cost of 7.